

Análisis de detección de reseñas falsas en plataformas de comercio electrónico con modelos de *Machine Learning*

- Cortes Moreno Mauricio Hamabiel, Pacheco Molina Miguel Alejandro, Maldonado López Alejandro, Soria Martínez Jesús Armando

Resumen – Se implementaron modelos de Machine Learning con técnicas de Procesamiento de Lenguaje Natural para clasificar reseñas falsas y auténticas en plataformas de comercio digital con el objetivo de observar el desempeño de cada algoritmo. El conjunto de datos utilizado consta de 20,000 reseñas generadas mediante una computadora y 20,000 reseñas originales humanas. Se realizó un preprocesamiento utilizando técnicas como normalización, tokenización y eliminación de stopwords. Asimismo, las reseñas fueron representadas mediante la técnica TF-IDF. Los modelos implementados son la regresión de Huber, Naive Bayes, SVM y Perceptrón Multicapa dando como resultado un modelo óptimo, la regresión de Huber. Se evaluó mediante métricas como accuracy, precisión, recall y F1-score, con un accuracy de 0.92 en promedio y F1-score de 0.92. Los resultados muestran la viabilidad de la aplicación de modelos de Machine Learning y técnicas de PLN para la detección de reseñas falsas en entornos de comercio electrónico.

Palabras Clave – Clasificación, Procesamiento de lenguaje natural, Reseñas falsas, TF-IDF, Regresión de Huber, Naive Bayes, SVM, Perceptrón Multicapa, Machine Learning.

1. Introducción

La clasificación de texto es una técnica de ciencia de datos que consiste en asignar automáticamente una etiqueta a un documento en función de su contenido. A diferencia del agrupamiento, que es un enfoque no supervisado, la clasificación requiere etiquetas conocidas para entrenar el modelo y evaluar su desempeño. Este tipo de técnicas es fundamental en aplicaciones como filtrado de spam, análisis de sentimiento, detección de fraudes, y clasificación de reseñas en comercio electrónico. En particular, la clasificación de reseñas generadas por computadora se ha vuelto relevante debido al crecimiento de contenidos automatizados, que pueden afectar la percepción de productos y servicios. En este nuevo paradigma digital, las opiniones y reseñas en línea se han convertido en uno de los activos más importantes para los compradores como para los vendedores. Las reseñas actúan como una validación social ya que influye de manera primordial en decisiones al momento de efectuar una compra por la manera de proveer información y experiencias. Para los vendedores representa una fuente fundamental de marketing para construir credibilidad y confianza. Para abordar este problema, es necesario combinar métodos de procesamiento de lenguaje natural (PLN) con modelos de machine learning que sean capaces de interpretar patrones sutiles en el texto.

En este contexto, las técnicas de representación de texto son fundamentales. Métodos como TF-IDF (**Term Frequency-Inverse Document Frequency**) demuestran ser altamente eficaces para la clasificación de texto. Este método permite que los algoritmos puedan cuantificar la relevancia de una palabra de un documento específico en relación con segmentos de documentos, capturando las señales lingüísticas que permiten diferenciar un texto de otro. Al momento de que el texto ha sido transformado en un formato numérico, es menester aplicar modelos eficaces de Machine Learning que serán entrenados para la tarea de clasificación binaria ya sea falsa o auténtica. Los modelos más eficaces explorados son **Naive Bayes**, **Maquina de Vectores de Soporte (SVM)** y **Perceptrón Multicapa (MLP)**, con la necesidad de comparar el desempeño de estos modelos en conjuntos de datos, especialmente aquellos que incluyen reseñas falsas generadas por computadora. Este enfoque permite identificar automáticamente reseñas generadas por computadora, evaluando su desempeño mediante métricas de clasificación estándar como accuracy, precisión, recall y F1-score, además de analizar la matriz de confusión para entender los tipos de errores. La combinación de PLN, vectorización y regresión robusta

proporciona un pipeline eficiente y escalable para la clasificación de reseñas, y puede ser extendido a otros tipos de textos automatizados

2. Objetivo

Aplicar técnicas de procesamiento de lenguaje natural y modelos de Machine Learning (Regresión de Huber, Naive Bayes, Máquinas de Vectores de Soporte y Perceptrón Multicapa), en conjunto con la técnica de vectorización TF-IDF, para la clasificación automática y eficaz de reseñas falsas y auténticas en el contexto del comercio electrónico.

2.1 Objetivos específicos:

- Limpiar y normalizar el texto de las reseñas mediante tokenización, eliminación de stopwords y reducción de palabras a su raíz (stemming/lemmatización).
- Transformar el texto limpio en vectores numéricos mediante TF-IDF, preparándolos para modelos de Machine Learning.
- Implementar y entrenar los cuatro modelos de clasificación de Machine Learning (Regresión de Huber, Naive Bayes, SVM y Perceptrón Multicapa) utilizando el conjunto de datos vectorizado.
- Evaluar el rendimiento de cada modelo entrenado utilizando un conjunto de métricas de clasificación estándar (Accuracy, Precisión, Recall y F1-Score) para cuantificar su capacidad predictiva.

3. Estado del arte

La detección de reseñas falsas o generadas por una computadora es uno de los principales desafíos en el campo del procesamiento del lenguaje natural (PLN). En los primeros estadios de la investigación, la detección de estas reseñas generadas por computadora se basaba en la ingeniería de características, un proceso tardado que intenta codificar heurísticas sobre la falsedad. No obstante, se experimentó, con la llegada de los modelos de aprendizaje máquina, un punto de inflexión en este campo; el estudio realizado por Ren y Zhang en el 2016 creó una nueva narrativa al proponer que la detección de reseñas falsas no solo reside únicamente en la elección de palabras, sino que se basa también en la misma estructura del discurso. El modelo que implementaron es uno de los primeros en ir más allá del simple análisis léxico para la automatización en detección de análisis de falsedad. Implementaron redes neuronales convolucionales para extraer información de las oraciones individuales y redes neuronales recurrentes para modelar las interrelaciones entre estas. En definitiva, su contribución es enseñar a la máquina a leer como un analista crítico [1].

Posteriormente, el conocimiento estaba fragmentado, se requería una vista panorámica para redefinir las fronteras del campo. El trabajo de Mohawesh et al. (2021) [2] actuó como el organizador deseado. Una nueva técnica que se basa en las representaciones contextuales que introdujo la arquitectura Transformer y modelos pre-entrenados como BERT. Estos modelos tienen la capacidad de generar representaciones de palabras que son dinámicas y dependen del contexto en el cual se desarrollan. Los autores propusieron un modelaje de ensamble que utiliza tres variantes de modelos transformer para la clasificación de reseñas, con esto maximizan el conocimiento del lenguaje obtenido.

Paralelamente con la evolución de los clasificadores de texto, surgió una línea de investigación complementaria que abordó el problema no como una tarea de clasificación directa, sino desde la detección de valores atípicos. El concepto de regresión, pero no como una herramienta de clasificación final. El trabajo de Mukherjee et al. (2012) [3] utilizó modelos de regresión de una manera ingeniosa para crear una distancia entre el sentimiento expresado en el texto estudiado y mediante la clasificación numérica dada.

De otra forma en [4], los autores desarrollaron un modelo de inteligencia artificial orientado a la clasificación y detección de cuentas bot y noticias falsas en Twitter (ahora X). Para lograrlo, implementaron redes neuronales recurrentes, junto con técnicas de procesamiento de lenguaje natural y análisis de sentimientos, utilizando herramientas como NLTK, TextBlob, spaCy y Botometer en Google Colab como entorno de desarrollo. El modelo fue entrenado con datos recolectados de Twitter, aplicando limpieza y preprocesamiento textual, esto permitió identificar patrones de comportamientos característicos de cuentas bot. Los resultados mostraron una precisión de 75.96% en la detección de cuentas automatizadas, lo que es un desempeño sólido del modelo para reconocer comportamiento no humano y fuentes de noticias falsas.

Otro trabajo es el que realizaron en [5], donde los autores desarrollaron un sistema destinado a identificar reseñas falsas o generadas por bots en plataformas de comercio electrónico, específicamente en Amazon, con el objetivo de verificar la autenticidad de la información que los usuarios utilizan para tomar decisiones de compra. Para lograrlo, implementaron un modelo de Machine Learning (ML) entrenado previamente, el cual aplica procesamiento de lenguaje natural a las reseñas y las clasifica como posibles reseñas SPAM o legítimas, junto con las probabilidades asociadas a dicho resultado. Los resultados mostraron que el sistema es capaz de distinguir de manera eficiente entre reseñas reales y artificiales, lo que la convierte en una herramienta útil que podrían usar los usuarios para mejorar la confianza en las plataformas de comercio electrónico.

4. Metodología

Para saber qué tipo de métodos se ocuparían se necesitaba explorar nuestro conjunto de datos. Es un conjunto sobre detección de reseñas falsas, disponible en Kaggle. El *corpus* Fake Reviews Dataset [10] contiene aproximadamente 40,000 reseñas de productos, de las cuales 20,000 son reales y 20,000 son falsas.

Donde este etiquetado como OR = *Original reviews* (reseñas creadas por humanos); CG = *Computer-generated fake reviews* (reseñas echas por bots)

	category	rating	label	text_
0	Home_and_Kitchen_5	5.0	CG	Love this! Well made, sturdy, and very comfor...
1	Home_and_Kitchen_5	5.0	CG	love it, a great upgrade from the original. I...
2	Home_and_Kitchen_5	5.0	CG	This pillow saved my back. I love the look and...
3	Home_and_Kitchen_5	1.0	CG	Missing information on how to use it, but it i...
4	Home_and_Kitchen_5	5.0	CG	Very nice set. Good quality. We have had the s...

Figura 1: Conjunto de datos crudo.

Al explorar el conjunto, se determinó que se ocuparían solo las columnas de **text_** y **label**, que serían la parte del texto y a etiqueta.

	text_	label
0	Love this! Well made, sturdy, and very comfor...	CG
1	love it, a great upgrade from the original. I...	CG
2	This pillow saved my back. I love the look and...	CG
3	Missing information on how to use it, but it i...	CG
4	Very nice set. Good quality. We have had the s...	CG
...	...	...
40427	I had read some reviews saying that this bra r...	OR
40428	I wasn't sure exactly what it would be. It is ...	CG
40429	You can wear the hood by itself, wear it with ...	OR
40430	I liked nothing about this dress. The only rea...	CG
40431	I work in the wedding industry and have to wor...	OR

Figura 2: Conjunto de datos filtrado.

Se agregó al conjunto la columna de **label_bin**, en donde la etiqueta se transformaba a binaria, convirtiendo CG = *Computer-generated fake reviews* y OR = *Original Reviews*, en 0 y 1 respectivamente.

	text_	label	label_bin
0	Love this! Well made, sturdy, and very comfor...	CG	0
1	love it, a great upgrade from the original. I...	CG	0
2	This pillow saved my back. I love the look and...	CG	0
3	Missing information on how to use it, but it i...	CG	0
4	Very nice set. Good quality. We have had the s...	CG	0
...	...	...	...
40427	I had read some reviews saying that this bra r...	OR	1
40428	I wasn't sure exactly what it would be. It is ...	CG	0
40429	You can wear the hood by itself, wear it with ...	OR	1
40430	I liked nothing about this dress. The only rea...	CG	0
40431	I work in the wedding industry and have to wor...	OR	1

Figura 3: Conjunto de datos transformado.

En el procesamiento de lenguaje natural, la forma de extraer, limpiar y preprocesar los datos es un tanto distinta pues presentan características particulares en comparación de los datos numéricos, pues se requiere trabajar con texto. Este trabajo implicó aplicar técnicas de normalización como:

- Normalizar todas las palabras a minúsculas con ayuda de funciones en Python como `lower()`.
- Eliminación de caracteres especiales con el uso de expresiones regulares como `[^a-z\s]`.
- Tokenización donde se divide un texto en unidades más pequeñas llamadas tokens. En este caso se hizo por palabras.
- Se eliminaron mediante funciones en Python las palabras vacías (stopwords) que son palabras muy comunes que no aportan un significado relevante para el análisis de un texto.

El procesamiento de lenguaje natural requiere transformar el texto en un formato estructurado y consistente que pueda ser interpretado por los algoritmos de *machine learning*. En este trabajo se realizaron dos procedimientos

de limpieza diferentes, dependiendo de la técnica de vectorización empleada se realizó una limpieza más estricta para el modelo *TF-IDF* y otra más ligera para los *embeddings* preentrenados *GloVe*.

Primero se aplicaron técnicas de normalización y limpieza sobre la columna *text_*, generando dos nuevas columnas en el conjunto de datos: *text_clean_tfidf*, con el texto preparado para la vectorización *TF-IDF*, y *text_clean_glove*, con el texto adaptado para la generación de *embeddings GloVe*.

Para la representación mediante *TF-IDF* esta representación era necesario eliminar la mayor cantidad de ruido posible, ya que *TF-IDF* se basa directamente en la frecuencia de aparición de las palabras. Las operaciones realizadas fueron las siguientes:

- Conversión a minúsculas mediante la función *lower()*, para evitar que palabras como “Good” y “good” se consideren diferentes.
- Eliminación de caracteres especiales con expresiones regulares (*re.sub(r'[^a-z\s]',' ', texto)*) para conservar solo letras y espacios.
- Tokenización que representa la división del texto en palabras individuales.
- Eliminación de stopwords o también conocidas como palabras vacías del inglés usando *nlk.corpus.stopwords*, ya que no aportan valor semántico.

El resultado fue una versión limpia del texto que conserva únicamente las palabras más relevantes, lista para ser transformada con el método *TF-IDF*.

En el caso de los *embeddings GloVe*, se aplicó una limpieza más ligera, ya que esta representación aprovecha el contexto y las coocurrencias de las palabras. Por ello, incluso cierta puntuación puede tener valor semántico.

Los pasos aplicados fueron los siguientes:

- Conversión a minúsculas.
- Eliminación de URLs mediante *re.sub(r"http\S+|www\S+", "", texto)*.
- Conservación de puntuación básica, eliminando solo símbolos extraños.
- Reducción de espacios múltiples para mantener el formato uniforme.

Así, se conservaron más características del lenguaje natural, manteniendo información contextual útil para los modelos de representación semántica como *GloVe*.

Finalmente, el conjunto de datos quedó con dos versiones procesadas del texto: una optimizada para la representación estadística con *TF-IDF* y otra más adecuada para *embeddings* densos con *GloVe*. Esto permite realizar una comparación más equilibrada entre ambos enfoques de vectorización y analizar cómo cada uno captura la información textual.

	text_	label	label_bin	text_clean_tfidf	text_clean_glove
0	Love this! Well made, sturdy, and very comfor...	CG	0	love well made sturdy comfortable love itvery ...	love this! well made, sturdy, and very comfort...
1	love it, a great upgrade from the original. I...	CG	0	love great upgrade original ive mine couple years	love it, a great upgrade from the original. f...
2	This pillow saved my back. I love the look and...	CG	0	pillow saved back love look feel pillow	this pillow saved my back. i love the look and...
3	Missing information on how to use it, but it i...	CG	0	missing information use great product price	missing information on how to use it, but it i...
4	Very nice set. Good quality. We have had the s...	CG	0	nice set good quality set two months	very nice set. good quality. we have had the s...
...	...	...	...	...	...
40427	I had read some reviews saying that this bra r...	OR	1	read reviews saying bra ran small ordered two ...	i had read some reviews saying that this bra r...
40428	I wasn't sure exactly what it would be. It is ...	CG	0	wasnt sure exactly would little large small si...	i wasn't sure exactly what it would be. it is ...
40429	You can wear the hood by itself, wear it with ...	OR	1	wear hood wear hood wear jacket without hood s...	you can wear the hood by itself, wear it with ...
40430	I liked nothing about this dress. The only rea...	CG	0	liked nothing dress reason gave stars ordered ...	i liked nothing about this dress. the only rea...
40431	I work in the wedding industry and have to wor...	OR	1	work wedding industry work long days feet outs...	i work in the wedding industry and have to wor...

Figura 4: Conjunto de datos limpio.

Una vez realizado el preprocesamiento, fue necesario transformar las reseñas en una representación numérica que los algoritmos de *machine learning* pudieran procesar.

Este paso conocido como vectorización del texto, permite cuantificar las características lingüísticas y semánticas de cada reseña para que los modelos puedan analizarlas.

En esta práctica se implementaron tres tipos de representaciones vectoriales:

1. *TF-IDF*
2. *TF-IDF enriquecido*
3. *embeddings GloVe*.

Cada uno de estos métodos ofrece una forma diferente de capturar la información del texto: mientras que TF-IDF se basa en la frecuencia de palabras, GloVe busca representar el significado semántico de las mismas mediante relaciones de contexto.

Primero se empleó *TF-IDF* (*Term Frequency – Inverse Document Frequency*), es una técnica que resalta la importancia de los términos más representativos dentro de cada documento en función de su frecuencia local y global.

TF-IDF es una combinación de *frecuencia de término (TF)* y *frecuencia inversa de documento (IDF)*. La frecuencia de término (TF) se basa en la suposición de que un término con una frecuencia más alta dentro de un documento se considera más importante que uno con frecuencia menor [11]. Y se calcula de la siguiente manera:

$$TF(t, d) = \frac{f_{t,d}}{\sum_k f_{k,d}}$$

Ecuación 1: Cálculo de la frecuencia de término (TF) para un término t en un documento d .

Donde:

- $f_{t,d}$ = número de veces que el término t aparece en el documento d .
- $\sum_k f_{k,d}$ = número total de términos en el documento d .

Según Zhiying et al. [11] solo depende del número de apariciones de un término específico en un documento local. Por lo tanto, la capacidad de TF para distinguir todos los documentos relevantes de otros documentos irrelevantes es muy baja debido a su desconocimiento de la frecuencia de recopilación. Para abordar este problema, tiempo después se propuso la frecuencia inversa de documentos (IDF) con una preocupación por la frecuencia de recopilación que mejoraba la capacidad discriminativa de un término para la clasificación de textos. IDF se extiende desde la frecuencia de documentos (DF), que significa el número de documentos donde aparece un término. Se propone sobre la base de la suposición de que un término que aparece en menos documentos se considera más importante que el que aparece en más documentos. El valor IDF de un término específico se puede obtener como se muestra a continuación:

$$IDF(t, d, D) = \log \frac{|D|}{DF(t, D)}$$

Ecuación 2: Cálculo de la frecuencia inversa de documento (IDF) para un término t en el conjunto de documentos D .

ó, en su versión optimizada para evitar divisiones por cero:

$$IDF(t, d, D) = \log \frac{|D| + 1}{DF(t, D) + 1}$$

Ecuación 3: Versión suavizada del cálculo de IDF.

Donde:

- $|D|$ = número total de documentos en el corpus.
- $DF(t, D)$ = número de documentos en los que aparece el término t .

En combinación, dio lugar al TD-IDF como:

$$TD - IDF(t, d, D) = TF(t, d) * IDF(t, d, D)$$

Ecuación 4: Cálculo del peso TF-IDF de un término t en un documento d dentro del conjunto de documentos D .

En donde, $TF - IDF(t, d, D)$ representa el peso del término t del documento d en el corpus D , mientras que $TF(t, d)$ representa el valor TF del término t en el documento d .

Para su implementación se ocupó la función `TfidfVectorizer()` de *sklearn* en Python donde ya está implementado este método de vectorización, se aplica TD-IDF.

```
vectorizer = TfidfVectorizer()
X_tfidf = vectorizer.fit_transform(df['text_clean'])

print("Forma del TF-IDF:", X_tfidf.shape)
```

→ Forma del TF-IDF: (40432, 48294)

Figura 5: Implementación de TD-IDF en Python.

Posteriormente, se aplicó una versión mejorada del método, denominada *TF-IDF enriquecido*. Esta variante mantiene el principio de ponderación entre frecuencia y rareza, pero con un preprocesamiento más detallado y un ajuste en los parámetros del vectorizador.

Entre los principales ajustes realizados se incluyeron:

- El uso de un rango más amplio de *n-gramas* (1, 2) para capturar combinaciones de palabras.
- La exclusión de términos excesivamente comunes o poco frecuentes mediante los parámetros *max_df* y *min_df*.

Con estas modificaciones, se buscó mejorar la capacidad discriminativa del modelo y capturar patrones más complejos en las reseñas, lo que resulta especialmente útil para distinguir entre textos generados por humanos y por computadora.

```
vectorizer_compuesto = TfidfVectorizer(  
    lowercase=True,  
    ngram_range=(1, 2),           #incluye unigramas y bigramas  
    max_df=0.9,                   #ignora términos que aparecen en >90% de docs  
    min_df=2,                     #ignora términos raros que aparecen <2 veces  
    sublinear_tf=True,            #usa log(1 + tf) -> reduce impacto de palabras repetidas  
)  
  
X_tfidf_compuesto = vectorizer_compuesto.fit_transform(df['text_clean_tfidf'])  
print("Forma del TF-IDF:", X_tfidf_compuesto.shape)  
  
Forma del TF-IDF: (40432, 133452)
```

Figura 6: Implementación de TD-IDF enriquecido en Python.

Finalmente, se utilizó la representación *GloVe* (*Global Vectors for Word Representation*), el cual es un modelo de *embeddings* preentrenado desarrollado por el grupo de investigación de *Stanford University* [10]. A diferencia de las representaciones basadas en frecuencias como TF-IDF, GloVe captura relaciones semánticas y contextuales entre palabras mediante el análisis estadístico de la coocurrencia de términos en un corpus extenso, lo que permite representar cada palabra como un vector denso en un espacio continuo de características.

En este proyecto se utilizó el modelo preentrenado GloVe de 50 dimensiones (*glove.6B.50d.txt*), entrenado sobre un corpus de 6 mil millones de palabras provenientes de Wikipedia y Gigaword.

Cada documento del corpus trabajado se representó promediando los vectores de las palabras que lo componen, generando así una representación numérica de dimensión 50 para cada texto.

Esta representación permitió capturar significados más ricos y relaciones semánticas entre los términos, mejorando la capacidad de los modelos de clasificación para identificar similitudes y contrastes conceptuales en los textos.

Para validar el correcto funcionamiento del modelo, se realizó una visualización 2D de los *embeddings* GloVe, donde se pueden observar agrupaciones de palabras semánticamente relacionadas y la distancia con términos contrastantes. La Figura 7 muestra dicha proyección, ilustrando cómo GloVe organiza las palabras en el espacio vectorial según su cercanía conceptual.

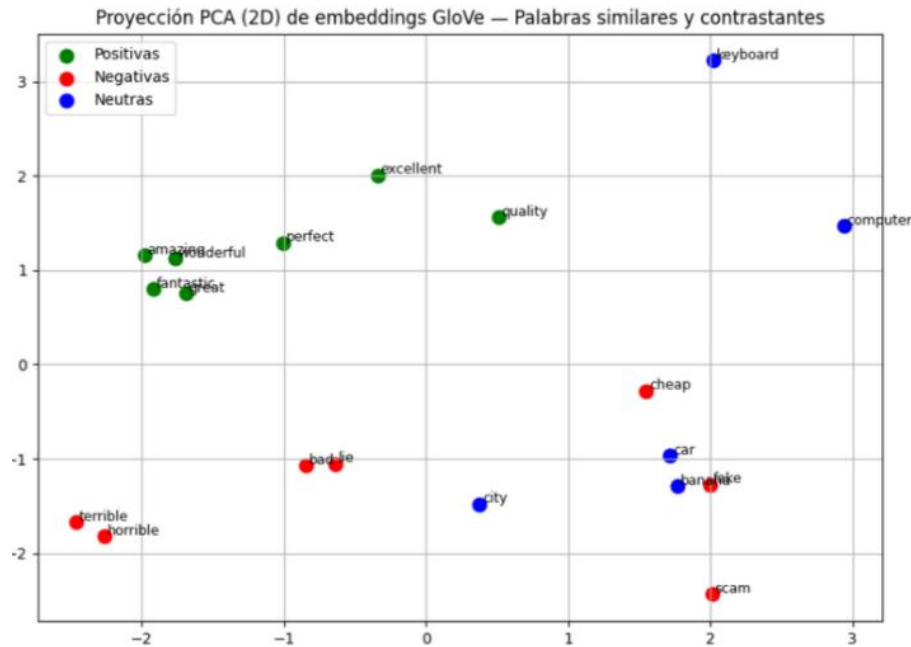


Figura 7: Visualización 2D de las palabras más cercanas y contrastantes según el embedding GloVe.

Se aplicaron diversos modelos de clasificación supervisada con el objetivo de predecir si una reseña fue redactada por un humano o generada por inteligencia artificial. Para ello, se emplearon las distintas representaciones vectoriales del texto previamente explicadas (*TF-IDF*, *TF-IDF enriquecido* y *GloVe*), combinándolas con modelos tanto lineales como no lineales. Esta estrategia permitió evaluar y comparar el desempeño, la estabilidad y la capacidad de generalización de cada modelo en diferentes configuraciones.

Para facilitar el entrenamiento, evaluación y comparación de los distintos modelos, se definió la función *modelado_entrenamiento()*. Esta función recibe como parámetros la matriz de características *X*, las etiquetas *y*, el modelo a entrenar, un nombre descriptivo del modelo y un umbral opcional para clasificación binaria.

En términos generales, la función realiza los siguientes pasos:

- Divide los datos en conjuntos de entrenamiento y prueba según proporciones predefinidas de entrenamiento y prueba (70-30%, 80-20% y 90-10%).
- Entrena el modelo con los datos de entrenamiento.
- Realiza predicciones sobre el conjunto de prueba y, si se especifica un umbral, ajusta la clasificación binaria de acuerdo con él.
- Calcula métricas de desempeño como *accuracy*, *F1 score*, *precision*, *recall* y el área bajo la curva ROC (*AUC*).
- Almacena los resultados en un *DataFrame* para su posterior análisis comparativo.
- Genera visualizaciones como matrices de confusión y curvas ROC para evaluar gráficamente el desempeño del modelo.

```
def modelado_entrenamiento(X, y, model, nombre_modelo, umbral = 0):
```

Figura 8: Definición de la función *modelado_entrenamiento()*, utilizada para estandarizar el entrenamiento, evaluación y comparación de los modelos de clasificación.

Como primera propuesta se empleó la *Regresión de Huber* como técnica de ajuste robusto. Este método fue introducido por *Peter Huber* en la década de 1960 como una alternativa a la regresión por mínimos cuadrados (*Ordinary Least Squares*, OLS), la cual es altamente sensible a valores atípicos y al ruido presente en los datos.

La regresión de Huber se basa en la función de pérdida de Huber, En está definida como:

$$\ell_{\sigma}(u) = \begin{cases} u^2, & \text{if } |u| \leq \sigma, \\ 2\sigma|u| - \sigma^2, & \text{if } |u| > \sigma, \end{cases}$$

Ecuación 5: Función de pérdida descrita por Wang et al. [11].

Donde $u = y_i - f(x_i)$ representa el error de la predicción y $\sigma > 0$ es el parámetro de robustez que controla el punto de transición entre el comportamiento cuadrático y el lineal. [11]

Esto hace que el método sea menos sensible a datos atípicos y ruidos con colas pesadas en comparación con OLS. Este modelo permite manejar la alta dimensionalidad de TF-IDF, reduciendo la influencia de outliers y ofreciendo coeficientes interpretables que facilitan el análisis de palabras predominantes. Aunque no es un clasificador puro, se fuerza a clasificar mediante un umbral.

Aplicando a nuestro caso, la regresión hará la predicción de la siguiente manera:

$$\hat{y} = \beta_0 + \sum_{i=1}^n \beta_i \cdot x_i$$

Ecuación 6: Función para hacer la estimación con Huber.

Donde:

- $\beta_0 = \text{sesgo}$
- $\beta_i = \text{coeficiente de la palabra } i$.
- $x_i = \text{valor TD - IDF de esa palabra en la reseña.}$

En Python, se definió el modelo utilizando la clase *HuberRegressor()* de scikit-learn, que permite un ajuste robusto frente a valores atípicos y ruido. Para entrenar y evaluar el modelo se empleó la función *modelado_entrenamiento()*, aplicando dos representaciones vectoriales: TF-IDF simple y TF-IDF compuesto.

```
X_huber = x_tfidf_simple
y_huber = df['label_bin']

huber = HuberRegressor()
modelado_entrenamiento(X_huber, y_huber, huber, 'Huber + TF-IDF simple', umbral = 0.5)

X_huber = x_tfidf_compuesto
y_huber = df['label_bin']
modelado_entrenamiento(X_huber, y_huber, huber, 'Huber + TF-IDF compuesto', umbral=0.5)
```

Figura 9: Entrenamiento del modelo Huber con el conjunto de entrenamiento.

Posteriormente, se exploraron los coeficientes de cada palabra y las distribuciones de las predicciones continuas generadas por el modelo. Esto permitió determinar un umbral adecuado para forzar la clasificación binaria, siguiendo la regla:

- Si $y_{pred} \geq 0.5$, el modelo predice la clase de reseña real (1).
- Si $y_{pred} < 0.5$, el modelo predice la clase de reseña falsa (0).

Como segunda propuesta se empleó el modelo *Naive Bayes*, un algoritmo de clasificación basado en la teoría de probabilidad de Bayes, ampliamente utilizado en tareas de procesamiento del lenguaje natural (PLN) y análisis de texto. Se denomina *naive* (ingenuo) porque asume que todas las características del conjunto de datos son independientes entre sí, una simplificación que rara vez se cumple en la práctica, pero que permite obtener resultados efectivos y eficientes.

Matemáticamente, el modelo se fundamenta en el *Teorema de Bayes*, el cual describe la probabilidad de que un texto pertenezca a una clase determinada C dado un conjunto de palabras X :

$$P(X / C) = \frac{P(X / C) P(C)}{P(X)}$$

Ecuación 7: Teorema de Bayes.

Donde:

- $P(C/X)$ es la probabilidad de que el texto pertenezca a la clase C dado el conjunto de palabras X .
- $P(X/C)$ representa la probabilidad de observar dichas palabras si el texto pertenece a esa clase.
- $P(C)$ es la probabilidad previa de la clase.
- $P(X)$ es la probabilidad total de observar ese conjunto de palabras.

El algoritmo calcula esta probabilidad para cada clase y asigna al texto aquella con el valor más alto. En este proyecto se utilizó la variante *MultinomialNB* de *scikit-learn*, comúnmente empleada en la clasificación de documentos, ya que considera la frecuencia de los términos como un factor relevante para la predicción.

El modelo se entrenó con las representaciones *TF-IDF* y *TF-IDF enriquecido*, con el objetivo de capturar patrones probabilísticos que permitan distinguir el lenguaje humano del generado por inteligencia artificial.

```
# Entrenamiento de Naive Bayes con TF-IDF simple
X_nb = X_tfidf_simple
y_nb = df['label_bin']
model_nb = MultinomialNB()
modelado_entrenamiento(X_nb, y_nb, model_nb, 'Naive Bayes + TF-IDF simple')

# Entrenamiento de Naive Bayes con TF-IDF compuesto
X_nb = X_tfidf_compuesto
modelado_entrenamiento(X_nb, y_nb, model_nb, 'Naive Bayes + TF-IDF compuesto')
```

Figura 10: Implementación del modelo *Naive Bayes* con las representaciones *TF-IDF simple* y *TF-IDF compuesto*.

Según McCallum y Nigam [10], la simplicidad y eficiencia de *Naive Bayes* lo convierten en un método base sólido para comparar el rendimiento de otros clasificadores más complejos en tareas de procesamiento del lenguaje natural.

Como tercera propuesta se utilizó el modelo *Support Vector Machine* (SVM), un algoritmo de clasificación supervisado ampliamente empleado para tareas de clasificación y detección de patrones en espacios de alta dimensionalidad. Su objetivo es encontrar un hiperplano óptimo que separe las clases dentro del espacio de características, maximizando el margen entre los datos de diferentes clases.

La ecuación general del hiperplano de separación se expresa como:

$$w^T x + b = 0$$

Ecuación 8: Hiperplano de separación del modelo SVM.

donde W representa el vector de pesos, x el vector de características y b el sesgo o término independiente. La función de decisión del modelo se define como:

$$f(x) = \text{sign}(w^T x + b)$$

Ecuación 9: Función de decisión del modelo SVM.

SVM resulta particularmente adecuado para representaciones vectoriales dispersas y de alta dimensión, como las generadas por *TF-IDF*, gracias a su capacidad para definir fronteras de decisión no lineales mediante el uso de funciones *kernel*. En este proyecto, se empleó la implementación de *sklearn.svm.SVC*, utilizando el kernel lineal debido a su buen rendimiento en datos textuales y su interpretabilidad.

El modelo se entrenó con las representaciones *TF-IDF enriquecido*, con el propósito de identificar los límites de separación entre reseñas reales y generadas por inteligencia artificial. Asimismo, se aplicó el modelo sobre las representaciones *GloVe* para evaluar su desempeño sobre embeddings densos. El modelo SVM es uno de los métodos más eficaces en tareas de *análisis de sentimientos* y *clasificación de texto*, debido a su robustez y capacidad para manejar datos complejos como señalan Manning, Raghavan y Schütze (2008) [11].

```
X_svm = X_tfidf_compuesto
y_svm = df['label_bin']

modelo_svm = SVC(kernel='linear', probability=True, random_state=42)
modelado_entrenamiento(X_svm, y_svm, modelo_svm, 'SVM + TF-IDF compuesto')

X_svm = X_glove
y_svm = df['label_bin']

modelo_svm = SVC(kernel='linear', random_state=7)
modelado_entrenamiento(X_svm, y_svm, modelo_svm, 'SVM + GloVe')
```

Figura 11: Implementación del modelo *SVM* con las representaciones *TF-IDF compuesto* y *GloVe*.

Como cuarta propuesta se utilizó el *Perceptrón Multicapa (MLP)*, un tipo de red neuronal que consiste en múltiples capas de neuronas interconectadas. Cada neurona aplica una función de activación a la suma ponderada de sus entradas, permitiendo al modelo capturar relaciones *no lineales complejas* entre las características de entrada y la salida.

Matemáticamente, la salida de una neurona en el MLP se define como:

$$y = \phi \left(\sum_{i=1}^n w_i x_i + b \right)$$

Ecuación 10: Salida de una neurona en el Perceptrón Multicapa.

Donde x_i son las entradas, w_i los pesos asociados, b el sesgo, y ϕ la función de activación.

En este proyecto, se entrenó el MLP con la representación de GloVe, explorando distintas combinaciones de funciones de activación y algoritmos de optimización para evaluar su impacto en el rendimiento del modelo.

Se consideraron las siguientes *funciones de activación*, cada una con características distintas:

- *Identity*: función lineal que devuelve el valor de entrada tal cual, útil para problemas de regresión y para comparar con funciones no lineales.
- *Logistic*: también conocida como sigmoide, comprime la salida entre 0 y 1, siendo adecuada para problemas de clasificación binaria.
- *Tanh*: función hiperbólica que transforma los valores de entrada en el rango $[-1, 1]$, ayudando a centrar los datos y mejorar la convergencia.
- *ReLU* (Rectified Linear Unit): devuelve 0 para entradas negativas y el valor de entrada para positivas, siendo muy utilizada por su eficiencia computacional y su capacidad para evitar el problema del gradiente desvanecido.

Asimismo, se probaron distintos *algoritmos de optimización*, que definen cómo se ajustan los pesos de la red durante el entrenamiento:

- *L-BFGS*: optimizador basado en métodos quasi-Newton, útil para problemas de pequeña a mediana escala y convergencia rápida.
- *SGD* (Stochastic Gradient Descent): realiza actualizaciones de los pesos usando un subconjunto aleatorio de los datos, lo que permite manejar grandes volúmenes de información y evita estancamiento en mínimos locales.
- *Adam*: combina ventajas de *SGD* con momentos adaptativos, ajustando dinámicamente la tasa de aprendizaje para cada parámetro y ofreciendo buena convergencia en problemas complejos.

La combinación de estas funciones de activación y optimizadores permitió explorar distintas configuraciones del MLP para capturar patrones complejos en las reseñas con la representación de los embeddings densos de GloVe.

Para la implementación en Python se utilizó la clase *MLPClassifier* de la librería *scikit-learn*, entrenando el modelo sobre las representaciones vectoriales y evaluando su desempeño mediante divisiones de entrenamiento

y prueba, siguiendo la misma estrategia de los demás modelos. Se realizando múltiples llamadas a la función *modelado_entrenamiento()*, variando la función de activación y el optimizador para cada experimento. Como ejemplo representativo, se muestra el entrenamiento del MLP con la función de activación *logistic* y el optimizador *L-BFGS* sobre los embeddings de GloVe:

```
X_mlp = X_glove
y_mlp = df['label_bin']

modelo_mlp = MLPClassifier(hidden_layer_sizes=(30,15), activation='logistic', solver='lbfgs', max_iter=500, random_state=7)
modelado_entrenamiento(X_mlp, y_mlp, modelo_mlp, 'Perceptrón Multicapa (logistic - lbfgs) + Glove')
```

Figura 12: Ejemplo de entrenamiento del Perceptrón Multicapa con función de activación *logistic* y optimizador *l-bfgs*.

A continuación, presentamos los resultados obtenidos con cada uno de los modelos y combinaciones de representaciones vectoriales, permitiendo evaluar su desempeño en la tarea de clasificación de reseñas.

5. Resultados

La regresión Huber con TF-IDF simple fue uno de los mejores modelos con un buen desempeño, obteniendo la siguiente evaluación:

- División de entrenamiento 90% y 10% prueba se obtuvo:

Métrica	Valor
Accuracy	0.8830
F1 Score	0.8835
Precision	0.8821
Recall	0.8885
AUC	0.9548
Verdaderos Negativos (TN)	1746
Falsos Positivos (FP)	244
Falsos Negativos (FN)	229
Verdaderos Positivos (TP)	1825

Tabla 1 : Resumen de métricas del modelo Huber + *TF-IDF* simple en la división 90/10.

Donde se observa una presión balanceada entre ambas clases y tanto el *recall* como *f1-score* favorece un poco a la clase de reseñas escritas por humanos. Y en la matriz de confusión se puede observar la poca presencia de falsos positivos y falsos negativos, con una curva *ROC* donde su *AUC* es de 0.94, indicando un buen rendimiento para distinguir las clases.

- División de entrenamiento 80% y 20% prueba se observó lo siguiente:

Métrica	Valor
Accuracy	0.8751
F1 Score	0.8768
Precision	0.8768
Recall	0.8826

AUC	0.9498
Verdaderos Negativos (TN)	3484
Falsos Positivos (FP)	532
Falsos Negativos (FN)	478
Verdaderos Positivos (TP)	3593

Tabla 2 : Resumen de métricas del modelo Huber + *TF-IDF* simple en la división 80/20.

Claramente se ve como en esta división el rendimiento fue un poco más bajo que el modelo anterior, pero aquí se observó un mejor rendimiento a favor a la clase a predecir del modelo, aunque bajo un poco su *AUC*, pero por poco, comparado con la división de 90/10.

- División de entrenamiento 70% y 30 % para prueba:
En este caso, el desempeño fue el siguiente:

Métrica	Valor
Accuracy	0.8768
F1 Score	0.8761
Precision	0.8748
Recall	0.8795
AUC	0.9477
Verdaderos Negativos (TN)	5303
Falsos Positivos (FP)	763
Falsos Negativos (FN)	731
Verdaderos Positivos (TP)	5333

Tabla 3 : Resumen de métricas del modelo Huber + *TF-IDF* simple en la división 70/30.

En este modelo no hubo una mejora significativa al primer modelo, y es casi similar el rendimiento al de división 80/20, lo que indica que Huber + *TF-IDF* se comporta mejor con una cantidad considerable de datos en entrenamiento.

Al igual, se hizo las mismas pruebas y divisiones, pero ahora implementando un *TF-IDF* más robusto, con bigramas, por lo que el modelo Huber + *TF-IDF* compuesto dio lo siguiente:

- División de entrenamiento 90% y 10% prueba:

Métrica	Valor
Accuracy	0.9236
F1 Score	0.9262
Precision	0.9090
Recall	0.9440
AUC	0.9744
Verdaderos Negativos (TN)	1796
Falsos Positivos (FP)	194
Falsos Negativos (FN)	115
Verdaderos Positivos (TP)	1939

Tabla 4: Resumen de métricas del modelo Huber + *TF-IDF* compuesto en la división 90/10.

En este caso, al aplicar una vectorización como TF-IDF agregando bigramas se observa un mejor rendimiento considerable, superando el 90% en todas las métricas y subiendo hasta un 0.97 en *AUC*.

- División de entrenamiento 80% y 20% prueba se observó lo siguiente:

Métrica	Valor
Accuracy	0.9211
F1 Score	0.9237
Precision	0.8997
Recall	0.9492
AUC	0.9545
Verdaderos Negativos (TN)	3585
Falsos Positivos (FP)	431
Falsos Negativos (FN)	207
Verdaderos Positivos (TP)	3864

Tabla 5: Resumen de métricas del modelo Huber + *TF-IDF* compuesto en la división 80/20.

Aquí se observa un rendimiento casi similar, pero no mejor al modelo con división de 90/10, ya que la proporción de falsos positivos con respecto a los falsos negativos se duplica, y baja un poco en la presión, lo que indica que es un buen desempeño, pero no el mejor en esta combinación de modelo + vectorización.

- División de entrenamiento 70% y 30 % para prueba:

Métrica	Valor
Accuracy	0.9183
F1 Score	0.9203
Precision	0.8981
Recall	0.9436
AUC	0.9736
Verdaderos Negativos (TN)	5417
Falsos Positivos (FP)	649
Falsos Negativos (FN)	342
Verdaderos Positivos (TP)	5722

Tabla 6 : Resumen de métricas del modelo Huber + *TF-IDF* compuesto en la división 70/30.

En esta división de entrenamiento/prueba no fue mejor que los otros dos, ya que no aumenta en las métricas el hecho de disminuir el porcentaje de datos para el entrenamiento del modelo, siendo el peor de los tres.

No obstante, se obtuvo un mejor rendimiento que en los modelos entrenados con TF-IDF simples, lo que indica la clave en la mejora fue en la aplicación de bigramas, llegando a la conclusión que el uso de TF-IDF considerando bigramas tiene una mejor generalización en los modelos, ya que ayuda a encontrar mejores patrones semánticos dentro de las reseñas.

Como se mencionó, también se implementaron clasificadores puros para observar el rendimiento en esta tarea, uno de ellos fue *Naive Bayes* que también fue sometido a los 3 diferentes tipos de entrenamiento/prueba y a los dos tipos de vectorización con TF-IDF.

Donde el desempeño de *Naive Bayes* + TF-IDF simple fue el siguiente:

- División de entrenamiento 90% y 10% prueba:

Métrica	Valor
Accuracy	0.8541
F1 Score	0.8475
Precision	0.9031
Recall	0.7984
AUC	0.8550
Verdaderos Negativos (TN)	1814
Falsos Positivos (FP)	176
Falsos Negativos (FN)	414
Verdaderos Positivos (TP)	1640

Tabla 7: Resumen de métricas del modelo *Naive Bayes + TF-IDF* simple en la división 90/10.

Al aplicar esta primera división de rendimiento, no hubo una mejora con respecto a los otros modelos, a primera instancia, la curva ROC recae hasta un 0.85, lo que la identificación entre casos no mejora.

- División de entrenamiento 80% y 20% prueba se observó lo siguiente:

Métrica	Valor
Accuracy	0.8564
F1 Score	0.8496
Precision	0.8986
Recall	0.8057
AUC	0.8568
Verdaderos Negativos (TN)	3646
Falsos Positivos (FP)	370
Falsos Negativos (FN)	791
Verdaderos Positivos (TP)	3280

Tabla 8: Resumen de métricas del modelo *Naive Bayes + TF-IDF* simple en la división 80/20.

Ya para la división 80/20 hubo una ligera mejora en comparación con la división 90/10, ya que aumenta sus métricas y curva ROC, pero no sigue siendo mejor que los modelos anteriores.

- División de entrenamiento 70% y 30 % para prueba:

Métrica	Valor
Accuracy	0.8572
F1 Score	0.8499
Precision	0.8958
Recall	0.8084
AUC	0.8572
Verdaderos Negativos (TN)	5496
Falsos Positivos (FP)	570
Falsos Negativos (FN)	1162

Verdaderos Positivos (TP)	4902
---------------------------	------

Tabla 9: Resumen de métricas del modelo *Naive Bayes + TF-IDF simple* en la división 70/30.

De las tres divisiones entrenamiento/prueba fue el mejor, dentro de este modelo, pero, sigue siendo peor que los anteriores, debido a que la proporción de falsos negativos era mayor en cada división, favoreciendo mucho a nuestra clase que no era nuestro objetivo, por lo que Naive Bayes + TF-IDF simple, no es de las mejores opciones para este caso.

Donde el desempeño de *Naive Bayes + TF-IDF* compuesto fue el siguiente:

- División de entrenamiento 90% y 10% prueba:

Métrica	Valor
Accuracy	0.8981
F1 Score	0.8984
Precision	0.9101
Recall	0.8870
AUC	0.8983
Verdaderos Negativos (TN)	1810
Falsos Positivos (FP)	180
Falsos Negativos (FN)	232
Verdaderos Positivos (TP)	1822

Tabla 10: Resumen de métricas del modelo *Naive Bayes + TF-IDF compuesta* en la división 90/10.

El uso de bigramas mejoró el desempeño del algoritmo de Naive Bayes, al punto de tener menos falsos positivos y falsos negativos, pero teniendo un comportamiento peculiar en la curva de *ROC*, con un *AUC* de 0.90, mejorando únicamente dentro del modelo Naive Bayes.

- División de entrenamiento 80% y 20% prueba se observó lo siguiente:

Métrica	Valor
Accuracy	0.9013
F1 Score	0.9014
Precision	0.9066
Recall	0.8963
AUC	0.9014
Verdaderos Negativos (TN)	3640
Falsos Positivos (FP)	376
Falsos Negativos (FN)	422
Verdaderos Positivos (TP)	3649

Tabla 11: Resumen de métricas del modelo *Naive Bayes + TF-IDF compuesta* en la división 80/20.

Para la división 80/20, hubo una mejora significativa en todas las métricas de evolución teniendo un resultado superior al 0.90 en todas las métricas, reducción de la proporción de los falsos positivos y falsos negativos, considerando que fue un buen desempeño.

- División de entrenamiento 70% y 30 % para prueba:

Métrica	Valor
Accuracy	0.8993
F1 Score	0.8987
Precision	0.9036
Recall	0.8938
AUC	0.8993
Verdaderos Negativos (TN)	5488
Falsos Positivos (FP)	578
Falsos Negativos (FN)	644
Verdaderos Positivos (TP)	5420

Tabla 12: Resumen de métricas del modelo *Naive Bayes* + *TF-IDF* compuesto en la división 70/30.

Al aumentar el porcentaje de entrenamiento no hay una mejora significativa en las métricas, con respecto a los otros dos modos de división, lo que se concluye que la división de entrenamiento 80/20 para el modelo Naive Bayes + TF-IDF compuesto es mejor, siendo este el mejor de los 6, incluyendo los modelos de Naive Bayes con el TF-IDF que solo está compuesto de bigramas, pero no sigue siendo mejor que Huber.

Para evaluar el rendimiento del modelo Support Vector Machine (SVM) con vectorización TF-IDF. El objetivo fue no solo medir la eficacia del clasificador, sino también evaluar su robustez y estabilidad ante diferentes volúmenes de datos de prueba.

Desempeño de *SVM* + TF-IDF compuesto fue el siguiente:

- División de entrenamiento 90% y 10% prueba:

Métrica	Valor
Accuracy	0.9214
F1 Score	0.9233
Precision	0.9153
Recall	0.9314
AUC	0.9212
Verdaderos Negativos (TN)	1813
Falsos Positivos (FP)	177
Falsos Negativos (FN)	141
Verdaderos Positivos (TP)	1913

Tabla 13: Resumen de métricas del modelo *SVM* + *TF-IDF* compuesta en la división 90/10.

La división 90-10 resultó en un modelo altamente preciso y equilibrado. El gran volumen de datos de entrenamiento permitió afinar su capacidad para evitar falsos positivos (la más alta precisión) sin dejar de ser bueno en la detección de los casos positivos reales (alto recall).

- División de entrenamiento 80% y 20% prueba se observó lo siguiente:

Métrica	Valor
Accuracy	0.9225
F1 Score	0.9244
Precision	0.9077

Recall	0.9418
AUC	0.9223
Verdaderos Negativos (TN)	3626
Falsos Positivos (FP)	390
Falsos Negativos (FN)	237
Verdaderos Positivos (TP)	3834

Tabla 14: Resumen de métricas del modelo *SVM + TF-IDF* compuesta en la división 80/20.

La división 80-20 creó el modelo con el mejor rendimiento general (F1-Score y AUC más altos). Su característica distintiva es su enfoque en el Recall.

División de entrenamiento 70% y 30 % para prueba:

Métrica	Valor
Accuracy	0.9192
F1 Score	0.9206
Precision	0.9052
Recall	0.9365
AUC	0.9192
Verdaderos Negativos (TN)	5471
Falsos Positivos (FP)	595
Falsos Negativos (FN)	385
Verdaderos Positivos (TP)	5679

Tabla 15: Resumen de métricas del modelo *SVM + TF-IDF* compuesto en la división 70/30.

La división 70-30 actuó como una prueba de estrés que el modelo superó con éxito. Demostró que el método es extremadamente robusto y no sufre una degradación significativa del rendimiento.

Desempeño de *SVM + Glove* compuesto fue el siguiente:

- División de entrenamiento 90% y 10% prueba:

Métrica	Valor
Accuracy	0.8207
F1 Score	0.8202
Precision	0.8358
Recall	0.8053
AUC	0.8210
Verdaderos Negativos (TN)	1665
Falsos Positivos (FP)	325
Falsos Negativos (FN)	400
Verdaderos Positivos (TP)	1654

Tabla 16: Resumen de métricas del modelo *SVM + Glove* compuesta en la división 90/10.

El 90% de entrenamiento creó el modelo más preciso y con el mejor rendimiento general, demostrando que un mayor volumen de datos (en este caso) ayuda a refinar la capacidad del modelo para evitar falsos positivos. Sin embargo, al comparar con las métricas del modelo *SVM + TF-IDF* se observa que no es mejor.

- División de entrenamiento 80% y 20% prueba se observó lo siguiente:

Métrica	Valor
Accuracy	0.8159
F1 Score	0.8144
Precision	0.8268
Recall	0.8023
AUC	0.8160
Verdaderos Negativos (TN)	3332
Falsos Positivos (FP)	684
Falsos Negativos (FN)	805
Verdaderos Positivos (TP)	3266

Tabla 17: Resumen de métricas del modelo *SVM + Glove* en la división 80/20.

Fue el modelo con el menor rendimiento, la diferencia es mínima (menos del 1%). Esto refuerza la idea de que el modelo SVM + Glove es muy estable. Sin embargo, la matriz de confusión nos indica que el modelo no es tan preciso al momento de realizar una clasificación, por lo que, este conjunto de entrenamiento y prueba no es ideal para esta combinación.

División de entrenamiento 70% y 30 % para prueba:

Métrica	Valor
Accuracy	0.8181
F1 Score	0.8159
Precision	0.8258
Recall	0.8062
AUC	0.8181
Verdaderos Negativos (TN)	5035
Falsos Positivos (FP)	1031
Falsos Negativos (FN)	1175
Verdaderos Positivos (TP)	4889

Tabla 18: Resumen de métricas del modelo *SVM + Glove* en la división 70/30.

La partición de 70-30 demuestra que el modelo es robusto, eficiente y generaliza en un rango aceptable, incluso cuando se entrena con un conjunto de datos más pequeño. Por lo que, en este caso no hay una diferencia muy grande al momento de hacer particiones con el conjunto de datos. No obstante, el modelo no es tan preciso y eficaz como el SVM + TF-IDF.

Perceptrón Multicapa:

Al igual, que, en los anteriores modelos, se entrenó un perceptrón multicapa unicamente con vectorización Glove, con distintos tipos de parámetros, diferentes combinaciones para obtener el más óptimo, los resultados fueron los siguientes:

12	Perceptrón Multicapa (identity - lbfgs) + Glove	90/10	0.859298	0.858704	0.876330	0.841772
13	Perceptrón Multicapa (identity - lbfgs) + Glove	80/20	0.855818	0.855478	0.863398	0.847703
14	Perceptrón Multicapa (identity - lbfgs) + Glove	70/30	0.853998	0.852699	0.860211	0.845317
15	Perceptrón Multicapa (identity - SGD) + Glove	90/10	0.856825	0.860381	0.852365	0.868549
16	Perceptrón Multicapa (identity - SGD) + Glove	80/20	0.850748	0.848500	0.867556	0.830263
17	Perceptrón Multicapa (identity - SGD) + Glove	70/30	0.853998	0.856425	0.842290	0.871042
18	Perceptrón Multicapa (logistic - lbfgs) + Glove	90/10	0.866469	0.866403	0.880785	0.852483
19	Perceptrón Multicapa (logistic - lbfgs) + Glove	80/20	0.864598	0.864060	0.873494	0.854827
20	Perceptrón Multicapa (logistic - lbfgs) + Glove	70/30	0.864963	0.863772	0.871309	0.856365
21	Perceptrón Multicapa (logistic - SGD) + Glove	90/10	0.415183	0.372513	0.409329	0.341772
22	Perceptrón Multicapa (logistic - SGD) + Glove	80/20	0.448745	0.170141	0.351268	0.112257
23	Perceptrón Multicapa (logistic - SGD) + Glove	70/30	0.499670	0.666043	0.499794	0.998021
24	Perceptrón Multicapa (tanh - lbfgs) + Glove	90/10	0.867211	0.867636	0.878682	0.856865
25	Perceptrón Multicapa (tanh - lbfgs) + Glove	80/20	0.863114	0.863821	0.865205	0.862442
26	Perceptrón Multicapa (tanh - lbfgs) + Glove	70/30	0.864963	0.864807	0.865664	0.863951
27	Perceptrón Multicapa (tanh - SGD) + Glove	90/10	0.865974	0.866830	0.875000	0.858812
28	Perceptrón Multicapa (tanh - SGD) + Glove	80/20	0.860393	0.858433	0.876793	0.840825
29	Perceptrón Multicapa (tanh - SGD) + Glove	70/30	0.860923	0.861551	0.857540	0.865600
30	Perceptrón Multicapa (relu - lbfgs) + Glove	90/10	0.862265	0.862095	0.877078	0.847614
31	Perceptrón Multicapa (relu - lbfgs) + Glove	80/20	0.860888	0.862149	0.860147	0.864161
32	Perceptrón Multicapa (relu - lbfgs) + Glove	70/30	0.862902	0.861521	0.870143	0.853067
33	Perceptrón Multicapa (relu - SGD) + Glove	90/10	0.659743	0.595056	0.752232	0.492210
34	Perceptrón Multicapa (relu - SGD) + Glove	80/20	0.664523	0.619655	0.721750	0.542864
35	Perceptrón Multicapa (relu - SGD) + Glove	70/30	0.661665	0.612025	0.717102	0.533806
36	Perceptrón Multicapa (relu - adam) + Glove	90/10	0.865480	0.865079	0.881699	0.849075
37	Perceptrón Multicapa (relu - adam) + Glove	80/20	0.857055	0.864827	0.825262	0.908376
38	Perceptrón Multicapa (relu - adam) + Glove	70/30	0.866529	0.862808	0.887398	0.839545

Figura 15: DataFrame comparativo de los distintos resultados del entrenamiento de todas las combinaciones posibles del Perceptrón Multicapa.

De todas las combinaciones, se eligió el perceptrón con mejor rendimiento que fue de la función de activación *logistic* y el *solver* lbfgs:

Perceptrón Multicapa (logistic + lbfgs) + Glove:

- División 90% entrenamiento y 10% prueba:

Métrica	Valor
Accuracy	0.8393
F1 Score	0.8391
Precision	0.8578
Recall	0.8194
AUC	0.8396
Verdaderos Negativos (TN)	1711
Falsos Positivos (FP)	279
Falsos Negativos (FN)	371
Verdaderos Positivos (TP)	1683

Tabla 19 : Resumen de métricas del modelo Perceptrón Multicapa (logistic + lbfgs) + Glove en la división 90/10.

El perceptrón multicapa con *logistic* y *lbfgs*, en esta primera división de entrenamiento tuvo un buen rendimiento, pero la precisión del modelo favorecía mejor a la clase de reseñas hechas por humanos, a la clase que no era nuestro objetivo, obteniendo una cantidad de falsos negativos considerable.

- División de entrenamiento 80% y 20% prueba se observó lo siguiente:

Métrica	Valor
Accuracy	0.8348
F1 Score	0.8334
Precision	0.8463
Recall	0.8209
AUC	0.8349
Verdaderos Negativos (TN)	3409
Falsos Positivos (FP)	607
Falsos Negativos (FN)	729
Verdaderos Positivos (TP)	3342

Tabla 20 : Resumen de métricas del modelo Perceptrón Multicapa (logistic + lbfgs) + Glove en la división 80/20.

En esta división (80/20) prácticamente se obtuvo el mismo desempeño, no hubo una mejora significativa o empeoro el modelo, se mantuvo casi igual, manteniendo el mismo problema en que tenía una cantidad considerable de falsos negativos por encima de otros modelos.

- División de entrenamiento 70% y 30% prueba se observó lo siguiente:

Métrica	Valor
Accuracy	0.8354
F1 Score	0.8337
Precision	0.8425
Recall	0.8250
AUC	0.8354

Verdaderos Negativos (TN)	5131
Falsos Positivos (FP)	935
Falsos Negativos (FN)	1061
Verdaderos Positivos (TP)	5003

Tabla 21: Resumen de métricas del Perceptrón Multicapa (*logistic + lbfgs*) + Glove en la división 70/30.

En las tres divisiones, el desempeño fue muy similar, aunque un poc mejor, pero fallaba mucho en identificar nuestra clase objetivo, ya que en los tres, hubo una proporción mayor que en otros modelos, y no hubo una mejora, por lo que el perceptrón multicapa (*logistic + lbfgs*) + Glove no fue una alternativa para mejorar el rendimiento respecto a modelos de clasificación y de regresión.

Conclusiones

Cortes Moreno Mauricio Hamabiel

El objetivo principal de esta práctica fue analizar distintos tipos de algoritmos para la detección de reseñas generadas por computador también se buscaba determinar si, más allá de las reglas lingüísticas, existían relaciones semánticas que aportaran valor al proceso de clasificación.

Para ello se utilizaron tres tipos de vectorización: TF-IDF con unigramas, TF-IDF con bigramas y una tercera forma basada en embeddings con GloVe, para el análisis de relaciones semánticas. Al aplicar TF-IDF con bigramas, se observó una mejora respecto a los modelos entrenados únicamente con TF-IDF simple, lo que permitió concluir que el uso de bigramas ayuda a capturar más reglas lingüísticas y expresiones compuestas relevantes para la clasificación, ya que muchos adjetivos o valoraciones suelen aparecer junto con una segunda palabra, como “muy bueno”, “me encantó”, o “lo odié”. Por último, al aplicar embeddings con GloVe, no se obtuvo una mejora significativa, ya que, análisis semántico más profundo no resultó ventajoso debido a la ambigüedad en los textos: existen humanos que escriben como bots y bots que escriben como humanos.

El mejor resultado se obtuvo con el modelo Huber junto con TF-IDF compuesto (bigramas), seguido del modelo SVM con la misma vectorización, lo que sugiere que, para este tipo de tareas, un análisis estadístico basado en características léxicas puede ser más que suficiente.

Maldonado López Alejandro

En definitiva, en este proyecto se aplicaron técnicas propias del procesamiento del lenguaje natural como TF-IDF y GloVe, modelos de aprendizaje máquina como lo son: Regresión de Huber, Máquina de vectores de soporte (SVM), Perceptrón Multicapa, Ingenuo de Bayes (Naive Bayes). Con el fin de clasificar reseñas de productos como generadas por una persona o generada por un humano. La limpieza y normalización del texto fueron implementados con precisión con el fin de una exitosa vectorización mediante TF-IDF e implementación de GloVe. Los modelos implementados obtuvieron un buen resultado mediante ambos métodos del procesamiento de lenguaje natural teniendo así que la regresión Huber es la más destacada ya que ayudó a manejar valores atípicos para generar predicciones más precisas. Con las predicciones se desarrolla una buena evaluación para la detección de las reseñas generadas por una computadora que pueden afectar enormemente a la industria comercial y de servicios digitales.

El desarrollo de nuestro proyecto nos permitió observar cómo la selección adecuada de técnicas de representación y modelado influye directamente en la eficacia de un sistema de clasificación textual. Los resultados indicaron que los modelos *Huber + TF-IDF* compuesto y *SVM + TF-IDF* compuesto alcanzaron los mejores desempeños, con precisiones superiores al 92% y valores de F1 cercanos a 0.94, superando notablemente a los enfoques basados en embeddings densos como GloVe. Esto demuestra que, en tareas donde las particularidades léxicas y de estilo son determinantes, las representaciones estadísticas como *TF-IDF* continúan ofreciendo una gran capacidad para distinguir patrones en el lenguaje.

Además, el trabajo destacó la importancia de cada etapa del flujo de aprendizaje automático, desde la limpieza y vectorización de los datos hasta la comparación y validación de los modelos, lo que contribuye al desarrollo de soluciones más sólidas y confiables. Los resultados obtenidos podrían aplicarse en distintos contextos prácticos, como la detección de reseñas falsas, el análisis de sentimientos, la clasificación de opiniones en redes sociales o la detección de correos spam, mostrando el potencial de estas técnicas en aplicaciones reales de procesamiento del lenguaje natural.

Soria Martínez Jesús Armando

Hoy más que nunca con el auge de inteligencia artificial y los modelos generadores de lenguaje, poder identificar que fue escrito por un humano y que, por un bot, resulta cada vez más difícil, tanto para personas individuales quienes están realizando una investigación y quieren fuentes confiables, así como también para las grandes empresas de comercio electrónico que se ven afectadas por comentarios falsos hechos por bots en ciertos productos o servicios, lo que les genera pérdidas económicas. Por todo esto este proyecto tiene aplicaciones importantes para poder clasificar los comentarios de manera automática.

El uso de técnicas de PLN fue fundamental para este proyecto, así como técnicas de aprendizaje automático, se entrenaron varios modelos, por un lado, con unigramas y bigramas se entrenó la regresión robusta de Huber y Naive Bayes, y por el otro lado se implementó Glove para entrenar SVM y perceptrón multicapa con diferentes combinaciones de función de activación y optimizadores. A pesar de que la Huber no se especializa en clasificación, con las adaptaciones necesarias cuando se entrenó con TF-IDF compuesto (bigramas), fue el modelo que obtuvo la mayor precisión.

7. Referencias

- [1] Ren, Y., & Zhang, Y. (2016, 1 diciembre). Deceptive opinion spam detection using neural network. ACL Anthology. <https://aclanthology.org/C16-1014/>
- [2] Mohawesh, R., Xu, S., Springer, M., Al-Hawawreh, M., & Maqsood, S. (2021). Fake or Genuine? Contextualised Text Representation for Fake Review Detection. Natural Language Processing. <https://doi.org/10.5121/csit.2021.112311>
- [3] Herzig, D. M., & Tran, T. (2012, 16 abril). Heterogeneous web data search using relevance-based on the fly data integration. 141-150. <https://doi.org/10.1145/2187836.2187856>
- [4] S. P. Castillo Cárdenas and F. J. Ávila Camacho, "Redes Neuronales Recurrentes para la detección de noticias falsas cuentas Bot en Twitter," *RICT Revista de Investigación Científica, Tecnológica e Innovación*, vol. 2, no. 4, pp. 64–71, Nov. 2024, doi: [10.5281/zenodo.14194198](https://doi.org/10.5281/zenodo.14194198)

- [5] P. Vilar Bustillo, *Sistema de detección automática de reseñas generadas artificialmente mediante aprendizaje automático*, Grado en Ingeniería Informática, Universidad de Cádiz, Cádiz, España, TFG, Jun. 2025. [En línea]. Disponible: <http://hdl.handle.net/10498/36585>
- [6] J. Salminen, C. Kandpal, A. M. Kamel, S. Jung, and B. J. Jansen, "Creating and detecting fake reviews of online products," *Journal of Retailing and Consumer Services*, vol. 64, p. 102771, 2022. doi: 10.1016/j.jretconser.2021.102771.
- [7] Z. Ji, B. Guo, Y. Huang, Y. Hu, P. Dwivedi, and Q. Zhu, "Text classification using novel term weighting scheme-based improved TF-IDF for Internet media reports," *Mathematical Problems in Engineering*, vol. 2021, Article ID 6619088, pp. 1–11, Mar. 2021, doi: 10.1155/2021/6619088.
- [8] Pennington, J., Socher, R., & Manning, C. D. (2014). *GloVe: Global Vectors for Word Representation*. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 1532–1543.
- [9] Y. Wang, B. Wang, C. Peng, X. Li, y H. Yin, "Huber Regression Analysis with a Semi-Supervised Method," *Mathematics*, vol. 10, no. 20, p. 3734, Oct. 2022. doi: 10.3390/math10203734.
- [10] A. McCallum, and K. Nigam, "A comparison of event models for Naive Bayes text classification," *Proceedings of the AAAI-98 Workshop on Learning for Text Categorization*, vol. 752, pp. 41–48, 1998.
- [11] Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press. <https://doi.org/10.1017/CBO9780511809071>